Relatório Final — API própria QWeb-compatível + Sync Informakon → CGStok

Data: 27/10/2026 · Banco analisado: `dbTREINO_CV` (treino) — produção = `dbCONV` (mesma estrutura).

1. O que foi entregue

Entregável	Status
`docs/qweb-contrato.md`	✅ Contrato da QWeb (36 endpoints catalogados, 5 observados live) + API de Integração REST :9090 (descoberta)
`docs/schema-informacom.md`	✅ Schema SQL Server do domínio de suprimento (PKs/FKs/colunas de auditoria)
`docs/schema-cgstok.md`	✅ Schema Prisma + adendo dos modelos de sync
`docs/mapeamento.md`	✅ Mapeamento campo a campo + esquema final implementado
`qweb-api/`	✅ Serviço Fastify+TS: rotas QWeb-compatíveis (XML), API almoxarifado `/estoques*` (JSON), `/supply/*` (debug), `/sync/*`, OpenAPI `/docs`, rate limit, API key, logs, cache TTL
Sync incremental	✅ Espelho `InfMirror` (hash) + projeção idempotente nas tabelas do CGStok, pendências persistentes (`projectedHash`), reconciliação por contagens
Migrations Postgres	✅ `informakon_sync` + `infmirror_projected_hash`
Feature flag	✅ `SystemSetting 'informakon.sync.enabled'`
`qweb-api/README.md`	✅ Instalação, envs, operação, deploy
`docs/qweb-api-documentacao.pdf`	✅ Documentação da API (gerada de `qweb-api-documentacao.html`)

2. Resultados da carga inicial (dbTREINO_CV)

Entidade	Linhas fonte	Projetadas	Projeção
unidade	84	84	`ProductUnit`
categoria	225	225	`ProductCategory` (Grupo_de_Insumos)
obra	4	4	`Worksite`

centro	47	47	`Warehouse` + local `*-GERAL`
insumo	22.325	22.325	`Product` (22 s em lote)
fornecedor	3.260	3.260	`Supplier`
funcionario	63	63	`User` (sem login)
estoque	2.115	2.115	`Stock`
entradaItem	4.396	4.396	`StockMovement` ENTRY (`INF-ENT-*`)
saidaItem	2.898	2.898	`StockMovement` EXIT (`INF-SAI-*`)

3. Contrato QWeb replicado

Rotas: `/versao`, `/empreendimentos`, `/clientes` (paginação obrigatória), `/getlistaobras`, `/validausuario`. Quirks preservados: HTTP 200 em erros de negócio (`<validacaoresultado>ERRO</validacaoresultado>`), XML ISO-8859-1, diacríticos removidos, `EnderecoCompleto` no formato `"End, Num - Bairro, Cidade - UF, Cep"`, `formato=json` ignorado. **Diferença proposital**: auth por `X-Api-Key` em tudo (QWeb expõe dados pessoais sem auth — não replicamos isso).

3.1 API do almoxarifado (`/estoques`)

Namespace próprio voltado ao domínio do CGStok (o termo "empreendimentos" ficou restrito à compat-QWeb): `GET /estoques` (almoxarifados com tipo, obra vinculada, total de insumos e quantidade) e `GET /estoques/{EMP.FIL.CENTRO}` (itens em estoque, mesmo formato do relatório oficial). Vínculo obra↔centro corrigido: `Empreendimento_CentrodeCusto.Centro_CodC` é código composto (`CVP010008`), não o centro simples — `Warehouse.worksiteId` agora populado.

3.2 Otimizações de desempenho

- **Cache TTL 30s** (env `CACHE_TTL_S`) nas GETs `/estoques*` e `/supply/*`:

HIT ≈ 1 ms vs ~37 ms; `?fresh=1` ignora; header `X-Cache`; stats em `/healthz`.

- **Sync passa a projetar só almoxarifados de obra**: centro de custo só vira `Warehouse` se for tipo obra (`EspCentro_Obra='S'` / `DM`) **com estoque real**, excluindo nomes administrativos (Stand/Receita/Marketing/Folha/Terreno...). Os demais centros foram inativados (o dropdown do CGStok já filtra ACTIVE).
- **Adoção de warehouses locais**: "Mahay Residence - Obra" e "Ousy - Obra" (criados no CGStok) foram adotados pelos `informakonKey` do ERP (`scripts/adopt-almoxarifados.cjs`) — o estoque sincronizado cai no almoxarifado operacional, sem duplicar. Nomes normalizados para "Mahay Residence"/"Ousy" (sufixo `- Obra (obr)` removido na projeção).
- Removido o UPDATE de `lastSeenAt` em linhas inalteradas (~22k escritas desnecessárias por ciclo no Postgres).
- Fetches de `estoque` / `funcionario` reescritos com `ROW_NUMBER()` (scan único em vez de MAX correlacionado por chave).
- Pool MSSQL `min: 1` (conexão quente; primeira request não paga handshake).

4. Descobertas de infraestrutura

- **QWeb** = CGI legado (Kiweb), porta 85, sem auth na maioria dos endpoints.
- **API de Integração REST** = WildFly :9090 `/informaconws/ws/` , Basic auth `integracao` , JSON, ~40 recursos — **todos comerciais** (nenhum de estoque).
- ⚠️ `177.22.35.89` expõe AMBAS na internet.
- SQL Server: nenhum `rowversion` utilizável nas tabelas de suprimento → estratégia hash + full-scan.
- `apidevin` está com `db_owner` /sysadmin (deveria ser `db_datareader`).

5. Riscos e pendências

1. 1. **Credenciais vazadas** em `InformakonApi/appsettings.json` (SQL + Twilio + AppInsights) e no e-mail (Basic auth :9090) → **rotacionar**.

1. 2. `apidevin` **com permissão de escrita** → pedir restrição ao DBA.
2. 3. `/clientes` **expõe CPF/RG/endereço sem auth** na QWeb e na nossa réplica

(com API key, melhor, mas ainda dados pessoais) → restringir se possível.

1. 4. **APIs expostas na internet** (`177.22.35.89:85` sem auth!) → firewall.
2. 5. `carteiraTitulosDetParcelas` não replicado (lógica de juros/multa com

retroatividade — fora do escopo do almoxarifado, documentado).

1. 6. Sync roda na máquina de dev — em produção, subir `qweb-api` como serviço

(systemd/NSSM) no servidor da rede interna (hoje: `npm start` via nohup;

não sobrevive a reboot).

1. 7. Latência da carga inicial (~25 min) — ciclos incrementais ~7-9 s.
2. 8. `disableRequestLogging` do Fastify emite deprecation warning (cosmético).
3. 9. As rotas `/supply/fornecedores` vs `/supply/credores` — decidir nome

canônico (hoje `/supply/credores` é o implementado).

1. 10. `CC:CJG.01.0002` ("Emp. Julio Gaspar - Stand de Vendas (construção)") tem

estoque real (89 itens) mas foi excluído do sync pelo filtro de nomes —

verificar se é o almoxarifado da obra do Metrô; se for, ajustar o nome no

ERP ou afrouxar a regra para a tag `(construção)`.

6. Próximos passos sugeridos

- Rodar contra `dbCONV` em produção (mesma estrutura; trocar `MSSQL_DATABASE`).
- Restringir `apidevin` a `db_datareader` + rotacionar credenciais.
- Fechar `177.22.35.89` ou autenticar a QWeb externa.
- UI no CGStok lendo `SyncCheckpoint` para "atualizado às HH:MM".
- Avaliar `/supply/*` como fonte de consulta ad-hoc dentro do app.